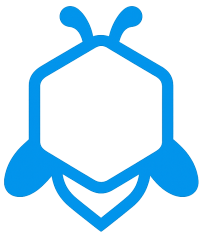
TaskHive

CSC457 - Web Tecnologies Project Report:
A Minimalist Task Management System



Students:
Mishari Albuhairi - Anas Muqtif

May 3, 2025

Contents

Chapter 1

Project Overview

1.1 Introduction

TaskHive is A Minimalist Task Management System designed to meet the needs of 80% of users with only 20% of the complexity, Our goal is to provide the most needed features and maintain the lowest possible learning curve

1.2 Students Information

Project Team Members

Student Name	UID	Contributions
Mishari Khalid Albuhairi	443102188	Frontend Development (React, UI Components), Docker Deployment
Anas Obaid Muqtif	443106490	Backend Development (PHP API), Database Design, User Authentication

Table 1.1: TaskHive Project Team Information

1.3 Website Link

https://taskhive.albuhairi.me/

Local installation instructions:

- Clone the repository
- Run `npm install` and `npm run dev` for the frontend
- Set up the PHP backend with Apache/MySQL with local server (XAMPP)

You can also run the Docker image directly

1.4 Website Description

Overview

TaskHive is a minimalist task management system designed to meet the needs of 80% of users with only 20% of the typical complexity. Our goal is to offer the most essential features while maintaining the lowest possible learning curve. The application enables users to create, organize, track, and complete tasks, with additional support features such as Kanban board visualization, subtask functionality, and an analytics dashboard

The primary target users are individuals who need an intuitive yet powerful system to manage their Tasks. TaskHive stands out with clean interface and multiple task visualization options.

1.4.1 Core Features

- **User Authentication System**: Registration, login, and secure session management
- **Task Management**: Create, update, and delete tasks with customizable statuses and priorities
- **Kanban Board**: Visual task organization with drag and drop functionality
- **Subtasks**: Break down complex tasks into manageable subtasks
- **Dashboard Analytics**: Visual representations of task distribution and completion rates
- **Responsive Design**: Functional interface across desktop and mobile devices

Chapter 2

Design

2.1 Design System

2.1.1 Color Palette

Light Theme

Element	Color Code	Usage
Primary Color	`#0284c7`	Buttons, links, and primary actions
Primary Light	`#38bdf8`	Hover states and secondary elements
Primary Dark	`#0369a1`	Active states and emphasis
Background	`#FFFFFF`	Main background color
Surface	`#F9FAFB`	Card backgrounds and secondary surfaces
Text Primary	`#111827`	Main text color
Text Secondary	`#6B7280`	Secondary text, labels, placeholders

Table 2.1: Light Theme Color Palette

Dark Theme

Element	Color Code	Usage
Primary Color	`#38bdf8`	Brighter in dark mode for contrast
Primary Light	`#7dd3fc`	Hover states in dark mode
Primary Dark	`#0284c7`	Active states in dark mode
Background	`#1e293b`	Main background in dark mode
Surface	`#374151`	Card backgrounds in dark mode
Text Primary	`#F9FAFB`	Main text in dark mode
Text Secondary	`#D1D5DB`	Secondary text in dark mode

Table 2.2: Dark Theme Color Palette

Status	Styling
Backlog	Gray (`bg-gray-100 text-gray-800`, `dark:bg-gray-700 dark:text-gray-300`)
Todo	Blue (`bg-blue-100 text-blue-800`, `dark:bg-blue-900/30 dark:text-blue-300`)
Doing	Purple (`bg-purple-100 text-purple-800`, `dark:bg-purple-900/30 dark:text-purple-300`)
Done	Green (`bg-green-100 text-green-800`, `dark:bg-green-900/30 dark:text-green-300`)

Table 2.3: Task Status Colors

Priority	Styling
High	Red (`bg-red-100 text-red-800`, `dark:bg-red-900/30 dark:text-red-300`)
Medium	Yellow (`bg-yellow-100 text-yellow-800`, `dark:bg-yellow-900/30 dark:text-yellow-300`)
Low	Green (`bg-green-100 text-green-800`, `dark:bg-green-900/30 dark:text-green-300`)

Table 2.4: Task Priority Colors

2.1.2 Functional Colors

Status Colors

Priority Colors

2.1.3 Typography

TaskHive uses a clean, modern typography system based on the Inter font family, a highly readable sans-serif typeface optimized for screen display.

Element	Specification
Font Family	Inter (sans-serif)
Base Font Size	16px (1rem)
Heading 1	1.5rem (24px) - `text-2xl`
Heading 2	1.25rem (20px) - `text-xl`
Heading 3	1.125rem (18px) - `text-lg`
Small Text	0.875rem (14px) - `text-sm`
Extra Small Text	0.75rem (12px) - `text-xs`

Table 2.5: Typography Specifications

2.1.4 UI Components

TaskHive's interface is built using a component-based approach, with consistent styling throughout the application.

Component	Specification
Primary Button	Sky Blue 600 background (`bg-primary-600`), white text, hover state Sky Blue 700 (`hover:bg-primary-700`)
Secondary Button	Transparent background with borders, dark text
Cards	8px border radius (`rounded-lg`), shadow for depth (`shadow`)
Form Elements	Full width inputs, consistent border radius, custom checkbox styling
Tables	Light gray headers, row dividers, responsive design

Table 2.6: UI Component Specifications

Element	Specification
Logo Concept	Stylized hexagon/honeycomb motif representing organization and community
Logo Sizes	512×512px, 256×256px (maskable), 192×192px
Icon System	React Icons library with Feather icon set (FiIcons)
Icon Style	Minimalist, consistent stroke width, modern aesthetic

Table 2.7: Logo and Icon Specifications

2.1.5 Logo and Icons

2.1.6 Responsive Design

TaskHive is fully responsive, adapting to different screen sizes using Tailwind CSS breakpoints:

Breakpoint	Size
Mobile	$<$ 640px
Small (sm)	$\geq$ 640px
Medium (md)	$\geq$ 768px
Large (lg)	$\geq$ 1024px
Extra Large (xl)	$\geq$ 1280px

Table 2.8: Responsive Design Breakpoints

2.1.7 Design Principles

The TaskHive design system is guided by the following core principles:

- **Minimalism:** Clean interfaces with imple design
- **Hierarchy:** Clear visual hierarchy through typography and color to guide users
- **Consistency:** Unified design language throughout the application
- **Contrast and Font size:** Color contrast and text sizing follow best practices
- **Responsiveness:** Adaptation to different screen sizes and devices

Chapter 3

Technical Implementation

3.1 Database Usage

The MySQL database design supports all core features:

Database Schema Components

1. **User Management**: The `users` table stores user account information, including usernames, email addresses, and securely hashed passwords.
2. **Task Organization**: The primary `tasks` table stores task details including title, description, status, priority, and due dates, with foreign key relationships to users.
3. **Status and Priority Management**: The `task_statuses` and `task_priorities` tables provide predefined options for task organization.
4. **Subtask**: The `subtasks` table allows for hierarchical task breakdown with parent child relationships.

The database design follows relational database best practices with proper foreign key constraints, indexes for performance optimization, and a normalized structure to prevent redundancy.

3.1.1 Database Schema

Below is the database schema that powers TaskHive:

Database Tables (1)

```sql
CREATE TABLE IF NOT EXISTS `users` (
  `user_id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(50) NOT NULL,
  `email` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `created_at` timestamp NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  PRIMARY KEY (`user_id`),
  UNIQUE KEY `username` (`username`),
  UNIQUE KEY `email` (`email`)
);

CREATE TABLE IF NOT EXISTS `task_statuses` (
  `status_id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(50) NOT NULL,
  `display_order` int(11) NOT NULL DEFAULT 0,
  PRIMARY KEY (`status_id`)
);

CREATE TABLE IF NOT EXISTS `task_priorities` (
  `priority_id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(50) NOT NULL,
  `display_order` int(11) NOT NULL DEFAULT 0,
  PRIMARY KEY (`priority_id`)
);

CREATE TABLE IF NOT EXISTS `tasks` (
  `task_id` int(11) NOT NULL AUTO_INCREMENT,
  `user_id` int(11) NOT NULL,
  `title` varchar(255) NOT NULL,
  `description` text DEFAULT NULL,
  `status_id` int(11) NOT NULL DEFAULT 1,
  `priority_id` int(11) DEFAULT 2,
  `due_date` datetime DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  PRIMARY KEY (`task_id`),
  KEY `user_id` (`user_id`),
  KEY `status_id` (`status_id`),
  KEY `priority_id` (`priority_id`),
  CONSTRAINT `tasks_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`) ON DELETE CASCADE,
  CONSTRAINT `tasks_ibfk_2` FOREIGN KEY (`status_id`) REFERENCES `task_statuses` (`status_id`),
  CONSTRAINT `tasks_ibfk_3` FOREIGN KEY (`priority_id`) REFERENCES `task_priorities` (`priority_id`)
);
```

Database Tables (2)

```
CREATE TABLE IF NOT EXISTS `subtasks` (
  `subtask_id` int(11) NOT NULL AUTO_INCREMENT,
  `task_id` int(11) NOT NULL,
  `title` varchar(255) NOT NULL,
  `is_completed` tinyint(1) DEFAULT 0,
  `created_at` timestamp NULL DEFAULT current_timestamp(),
  `updated_at` timestamp NULL DEFAULT current_timestamp() ON UPDATE
    ↪ current_timestamp(),
  PRIMARY KEY (`subtask_id`),
  KEY `task_id` (`task_id`),
  CONSTRAINT `subtasks_ibfk_1` FOREIGN KEY (`task_id`) REFERENCES `
    ↪ tasks` (`task_id`) ON DELETE CASCADE
);
```

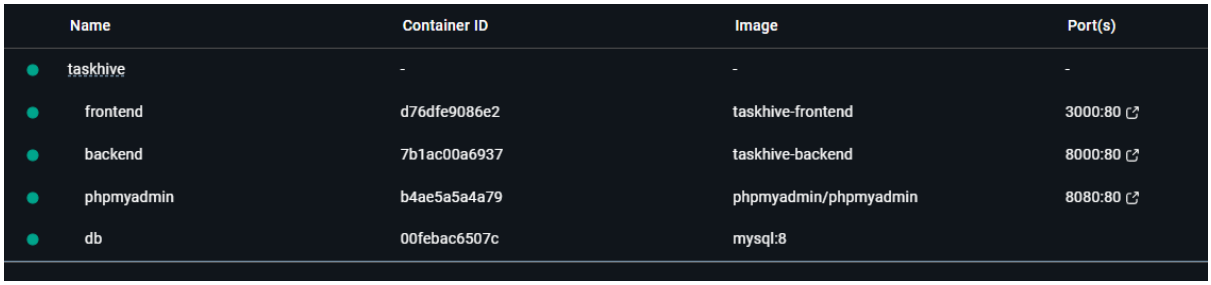
3.2 Used Website Technologies

3.2.1 Frontend

- **React**: Used as the primary frontend library
- **Vite**: Used as the build tool for more optimized production build than Create React App (CRA)
- **Tailwind CSS**: Used for styling with a utility classes that help in styling, like theme switching, tables design, and responsive layout
- **React Router**: For client side routing and navigation
- **Axios**: Used for making HTTP requests to the backend API
- **React Icons**: Integrated for icon system
- **Framer Motion**: Applied for smooth animations and transitions (Used in Kanban board)

3.2.2 Backend

- **PHP**: Used for building the RESTful API endpoints
- **MySQL**: Employed as the relational database management system
- **Docker**: Used for containerization by packaging the frontend, backend, MySQL database, and phpMyAdmin into isolated containers, each containing the OS, code, and all necessary dependencies to run consistently on any server



Name	Container ID	Image	Port(s)
taskhive	-	-	-
frontend	d76dfe9086e2	taskhive-frontend	3000:80
backend	7b1ac00a6937	taskhive-backend	8000:80
phpmyadmin	b4ae5a5a4a79	phpmyadmin/phpmyadmin	8080:80
db	00febac6507c	mysql:8	

Figure 3.1: Docker Containers

3.2.3 Dependency Management

Package.json Configuration

```
{
  "name": "taskhive-frontend",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
    "@craco/craco": "^7.1.0",
    "@testing-library/jest-dom": "^5.16.5",
    "@testing-library/react": "^13.4.0",
    "@testing-library/user-event": "^13.5.0",
    "axios": "^1.9.0",
    "framer-motion": "^10.18.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-icons": "^4.12.0",
    "react-router-dom": "^6.30.0",
    "react-scripts": "5.0.1",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.4.1",
    "autoprefixer": "^10.4.21",
    "cross-env": "^7.0.3",
    "postcss": "^8.5.3",
    "tailwindcss": "^3.4.17",
    "vite": "^6.3.4"
  },
  "proxy": "https://api-taskhive.albuhairi.me/"
}
```

3.3 Technical Overview

3.3.1 Frontend Architecture

- Built with React using functional components and hooks
- State management through React Context API (AuthContext for user authentication)
- Responsive layouts using Tailwind CSS utility classes
- Client side routing with React Router

3.3.2 Backend Architecture

- RESTful API built with PHP
- Secure authentication with password hashing and using tokens
- Database access through PDO to prevent SQL injection attacks
- Modular code organization with utilities for common functions

3.3.3 Client-Server Communication

- Frontend communicates with backend exclusively through REST API calls
- JSON data format for all requests and responses
- Token based authentication passed via the request
- proper error handling and status codes

Chapter 4

Application Screenshots

4.1 Login Page

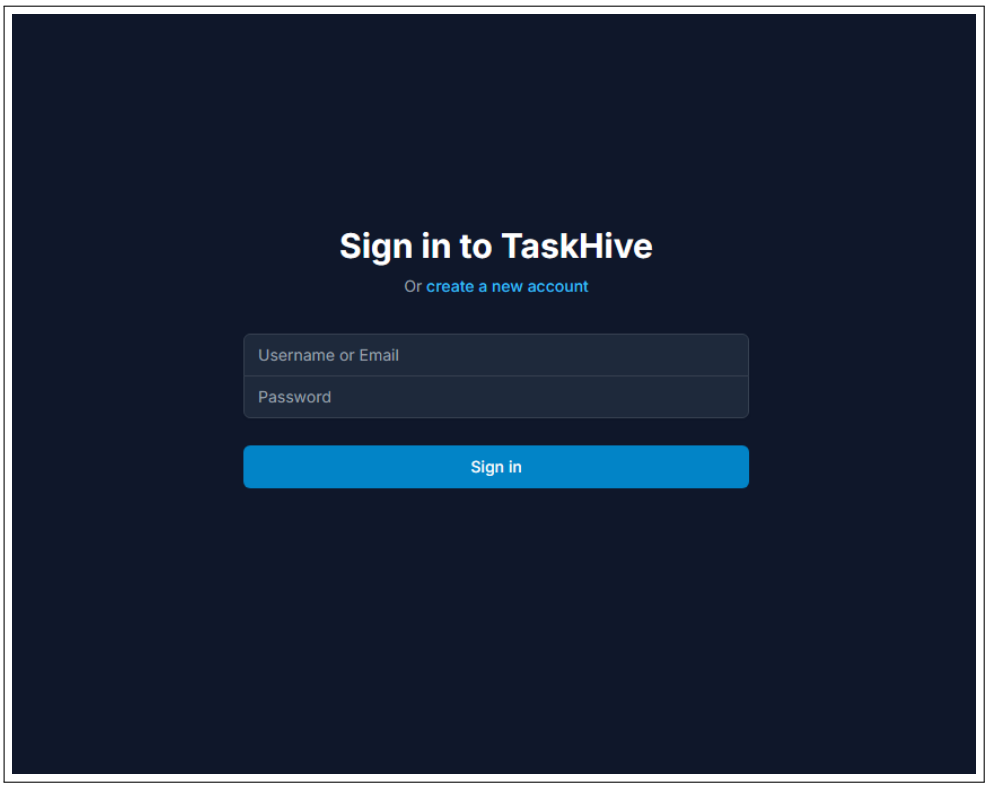


Figure 4.1: The login page provides secure authentication for users to access their tasks.

4.2 Registration Page

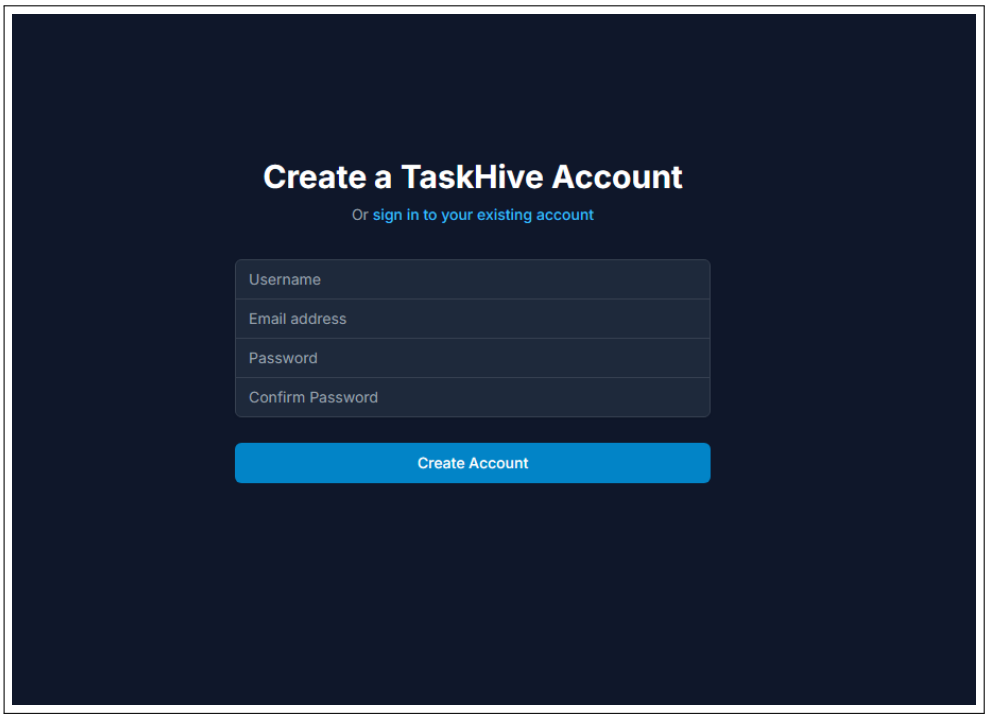


Figure 4.2: New users can create accounts with basic information.

4.3 Dashboard

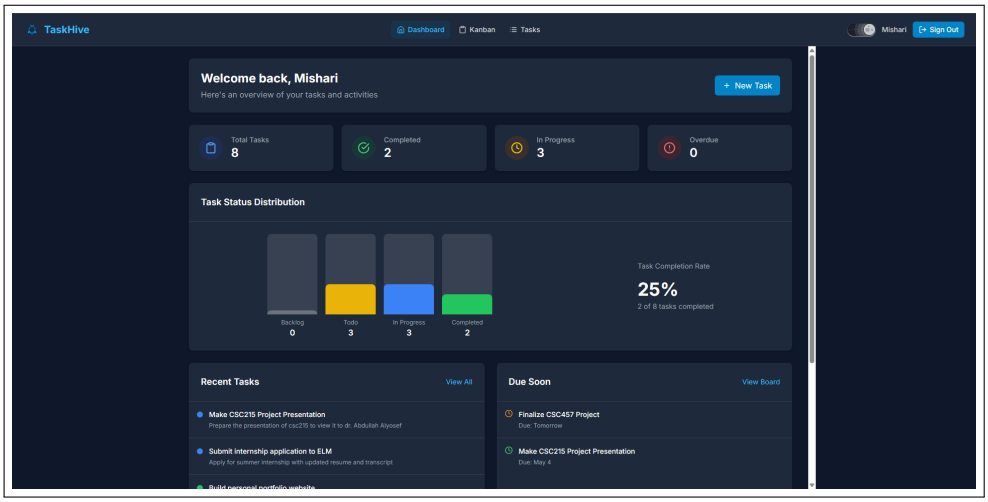


Figure 4.3: The dashboard provides an overview of task status distribution and completion metrics.

4.4 Task List View

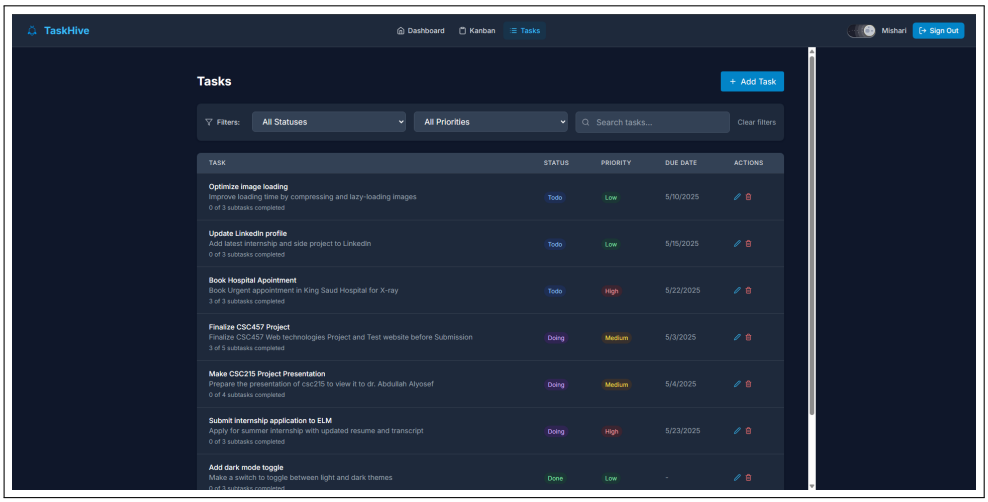


Figure 4.4: The list view displays all tasks with filtering and sorting options.

4.5 Task Details

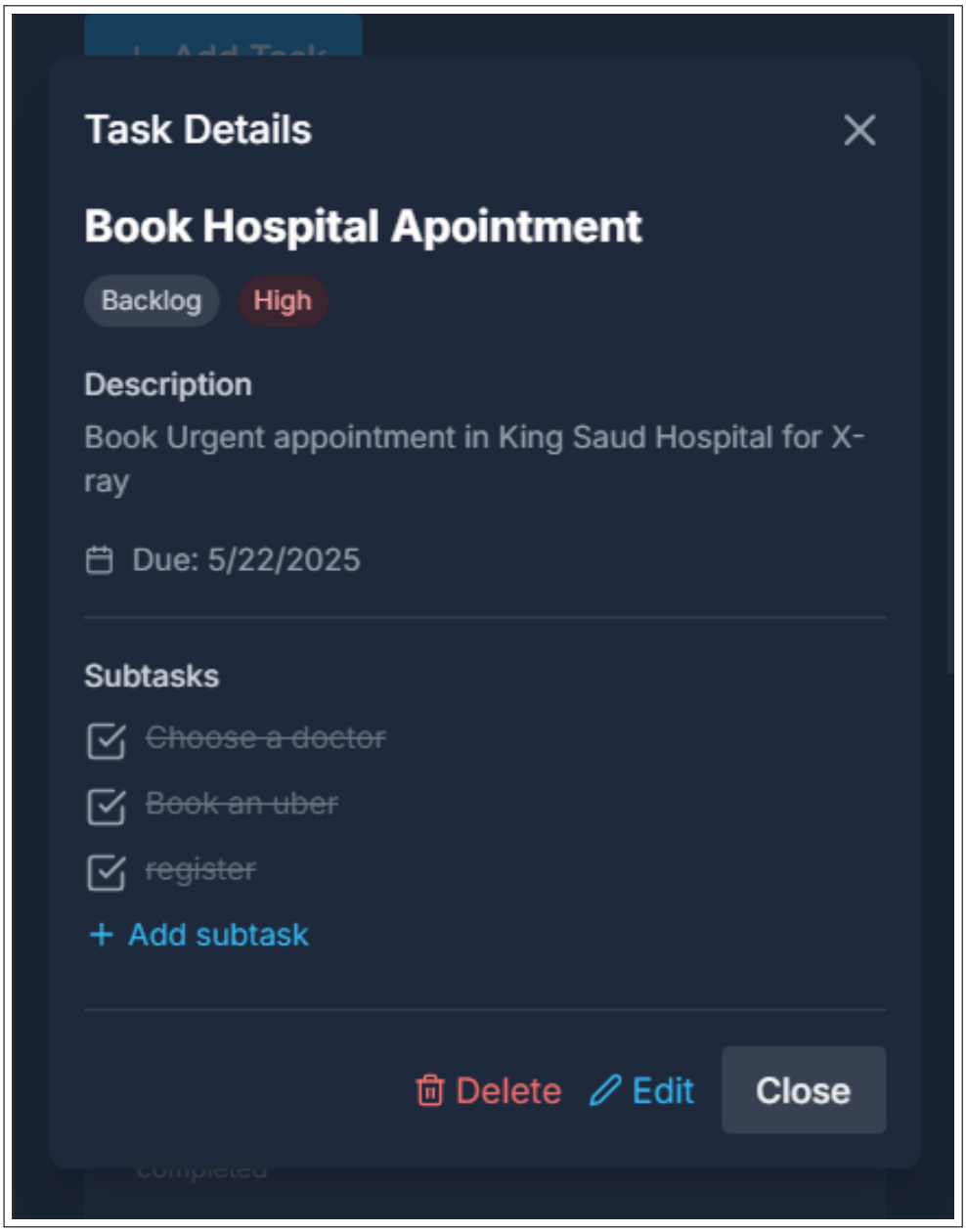


Figure 4.5: Users can view and edit detailed information about each task, including subtasks.

4.6 Kanban Board

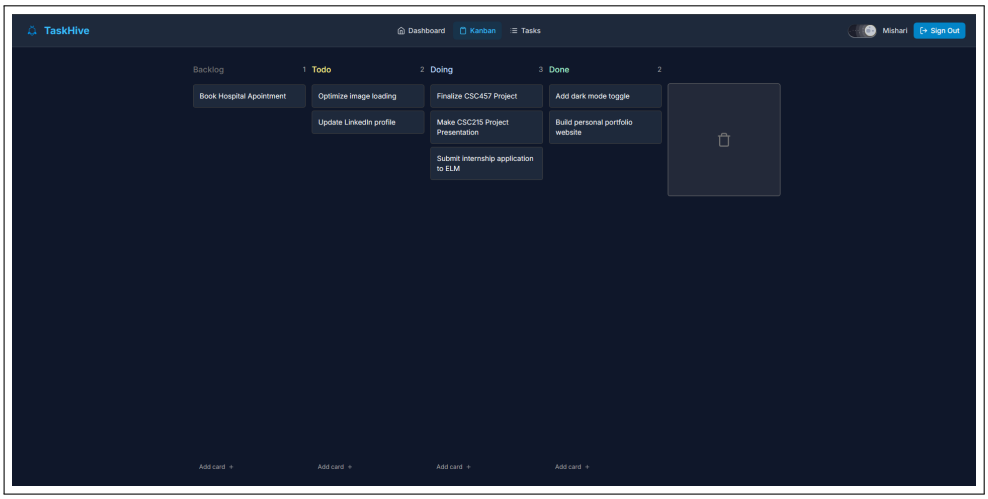


Figure 4.6: The kanban board offers a visual workflow representation with drag-and-drop functionality.

4.7 Create/Edit Task Form

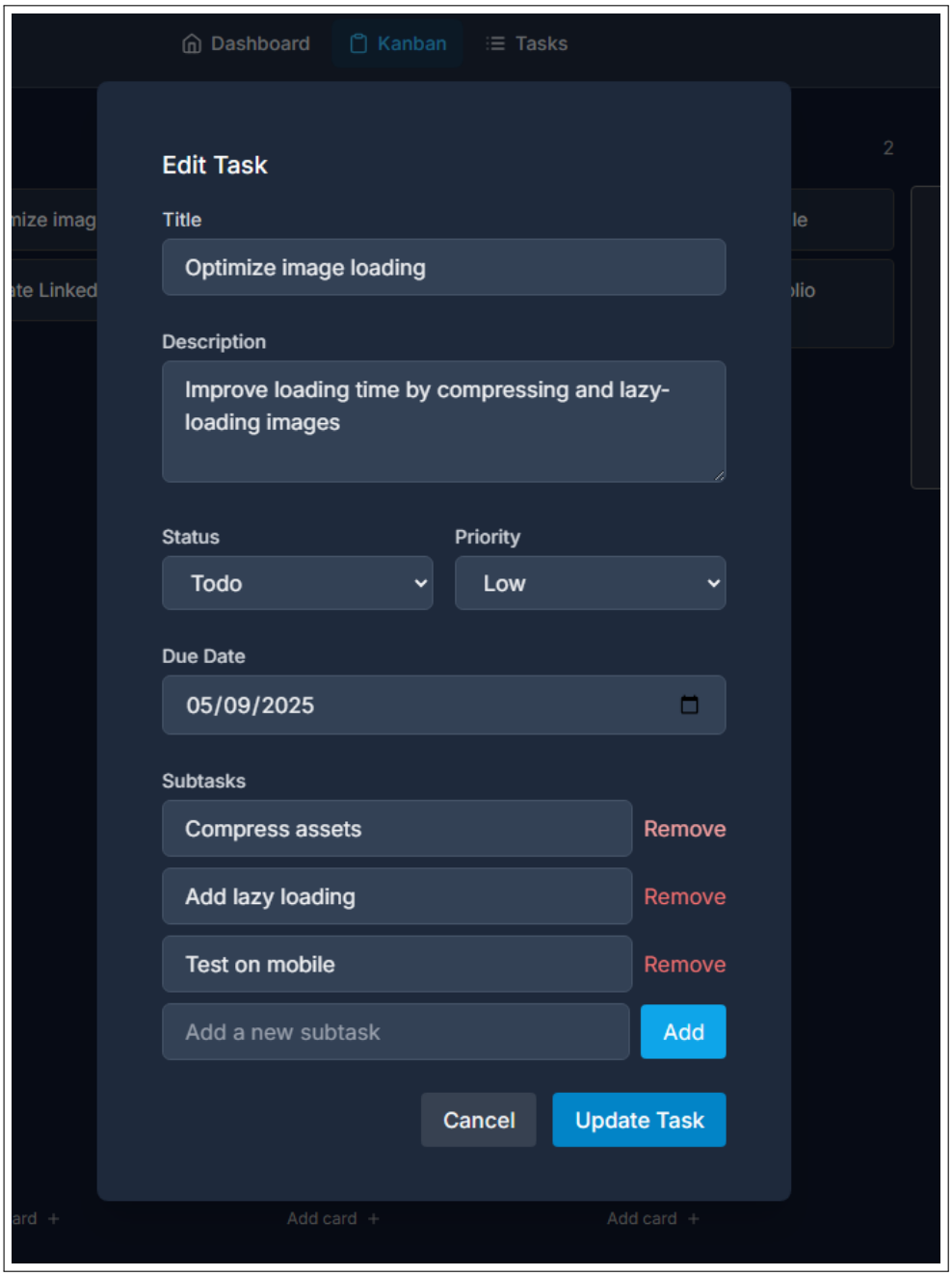


Figure 4.7: Users can create new tasks or modify existing ones with all relevant details.

4.8 Mobile View

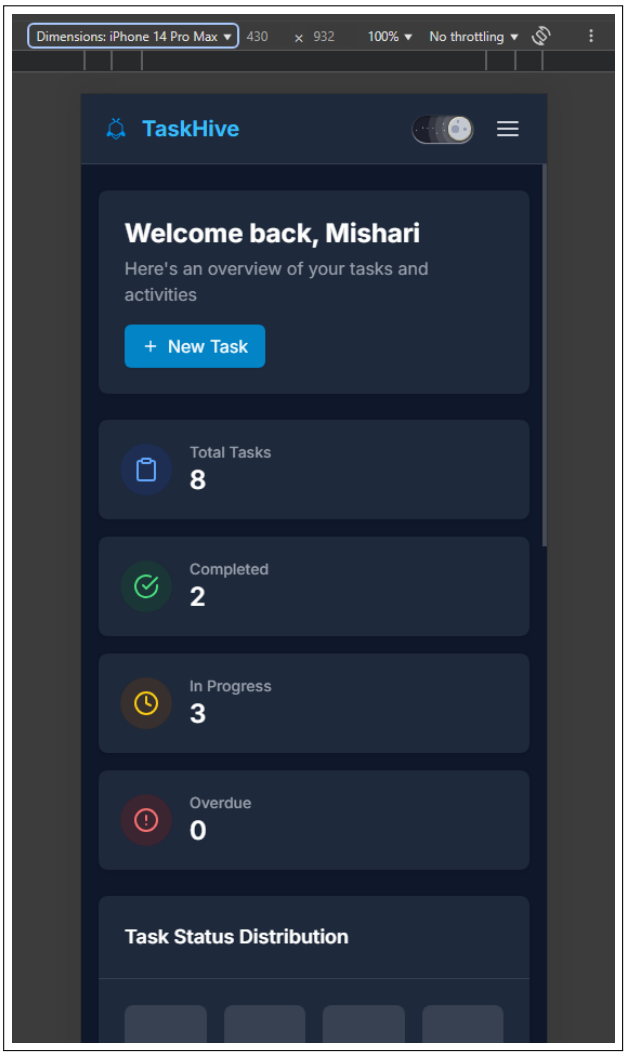


Figure 4.8: TaskHive is fully responsive and have seamless experience on mobile devices.